北京智源人工智能研究院
BEIJING ACADEMY OF ARTIFICIAL INTELLIGENCE

KernelGen：Let AI write its own kernels

Dongxu Han

Beijing Academy of Artificial Intelligence (BAAI)

10/19/2026

What AI is doing when you talking with it

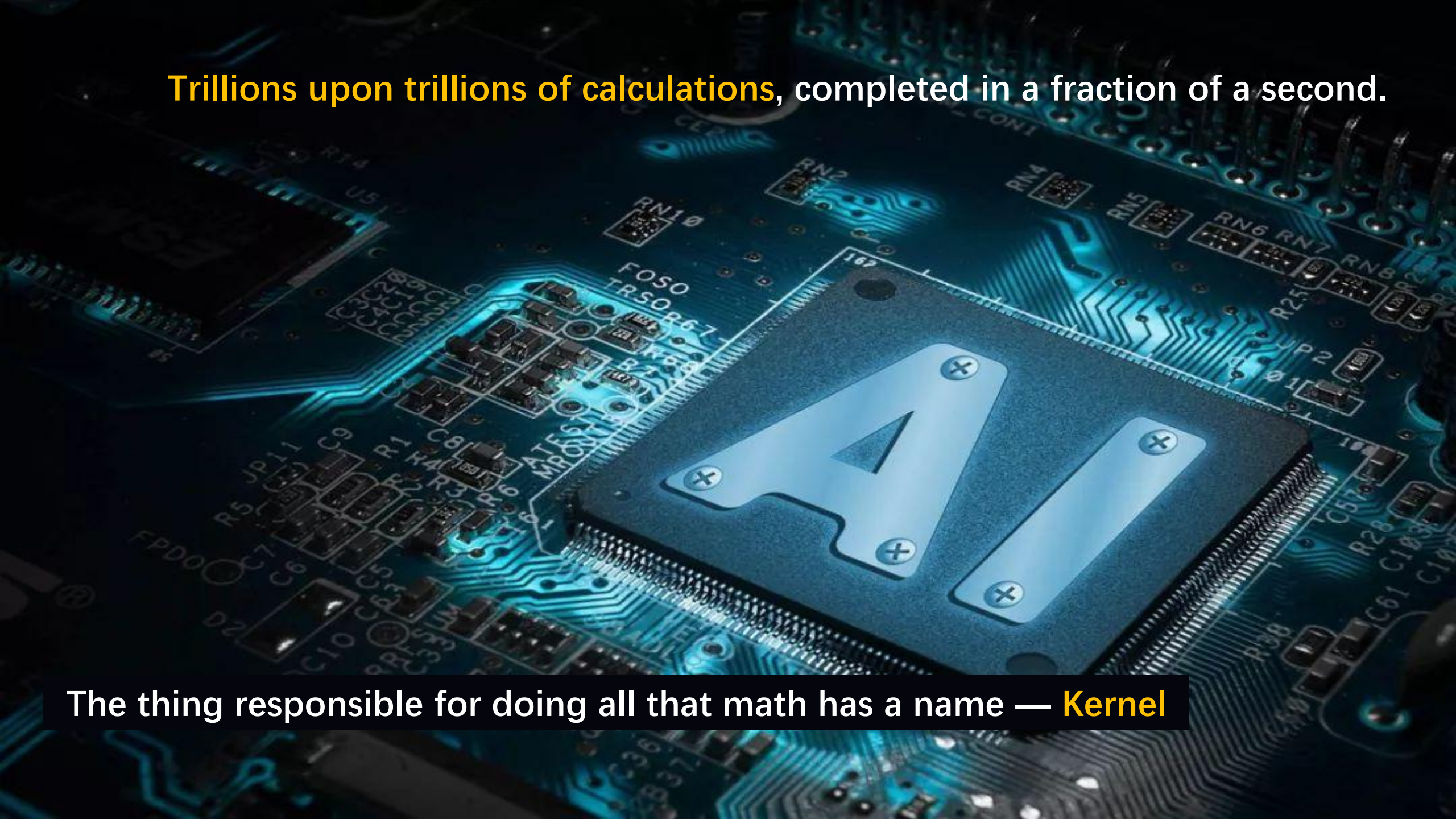











Trillions upon trillions of calculations, completed in a fraction of a second.
AI
The thing responsible for doing all that math has a name — Kernel

Understanding AI Through a Restaurant Analogy

A large AI model is like a big restaurant

MODEL = THE MENU

Hundreds of dishes listed

CHIP = THE STOVE

The powerful kitchen equipment

KERNEL = COOKING INSTRUCTIONS

How to chop, how to stir-fry,
what temperature,
when to add salt



Without cooking instructions,

even the finest ingredients and the most expensive stove can't produce a single dish.

Who Writes These Kernels?

A very small group of expert engineers



Developing an Operator is a Complex Process

Support various shapes, dtypes, and achieve high performance

Shape Support

- ✓ Different batch sizes
- ✓ Different sequence lengths
- ✓ Different tensor dimensions
- ✓ Dynamic shapes

B, H, W, C

B, N, C

B, S, D

B, C, H, W

...

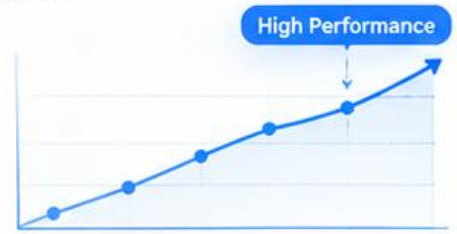
Dtype Support

FP32	FP16	BF16	INT8
INT4	UINT8	BOOL	...

```
@triton.jit
def kernel(...):
    pid = tl.program_id(0)
    off = pid * BLOCK_SIZE
    for i in range(0, n, BLOCK_SIZE):
        x = tl.load(...)
        y = tl.dot(...)
        tl.store(..., y)
```

Performance Optimization

- ✓ Memory Access
- ✓ Computation Efficiency
- ✓ Tiling & Blocking
- ✓ Kernel Tuning
- ✓ Cross-chip Adaptation



Tiling



...

NVIDIA → Ascend → Hygon

Moore Threads → MetaX → TianShu ...

- ✓ Correctness tests (pass)
- ✓ Performance benchmark (pass)
- ✓ Cross-shape validation (pass)
- ✓ Cross-dtype validation (pass)
- ✓ Cross-chip validation (pass)

CUDA / Triton

Deep Learning

Computer Architecture

Shape ✓
Dtype ✓
Performance ?
Cross-chip ✓

Time Consuming

Days to Weeks

From design, implementation, testing to optimization...

One Model = Hundreds of Operators

Each operator plays a critical role

Model

- Embedding — 12 ops
- Attention — 28 ops
- MLP — 34 ops
- Normalization — 16 ops
- Activation — 12 ops
- MoE — 18 ops
- More

100+ Operators



Operator Library

- MatMul
- Conv2d
- LayerNorm
- RMSNorm
- Gemm
- Attention
- Softmax
- SiLU
- Add
- Multiply
- Reshape
- Transpose
- More Operators

Attention → MLP → Norm → Activation → ...

Complex Model

Hundreds of operators, working together

One Model, Many Chips

Adapting to different chips brings significant migration effort



The Problem: Writing Kernels Is Incredibly Hard

北京智源人工智能研究院
BEIJING ACADEMY OF ARTIFICIAL INTELLIGENCE

Days to Weeks	Hundreds of Kernels	Rewrite from Scratch
One Kernel	One Model	Switch Chip

Models are getting bigger, Chips are multiplying,
But there aren't nearly enough people to write kernels

The ceiling of AI code generation

AI-generated code has become commonplace.

- A wide range of AI coding assistants: Claude, Cursor, GitHub Copilot
- AI coding lowers the coding barrier: writing code becomes a conversation with AI.

AI can write code, but it cannot write high-performance code.

- Ask AI to build a webpage, and it can produce one quickly.
- Ask AI to optimize an operator to hit the performance limits of a GPU, and the story is entirely different:
  - How to leverage hardware capabilities
  - How to implement parallel computing
  - How to reduce memory access

  This is not a programming language problem; it is an engineering problem.
  And kernels happen to fall into the **most challenging** category of engineering work.

The next phase of AI coding is unfolding at the kernel level.

Kernel Generation ≠ Code Generation

Kernel is not a one-time code generation problem,
but a closed-loop optimization process

Code Generatiom

Write code once, get the result

Generate → Syntax → Correct → Done





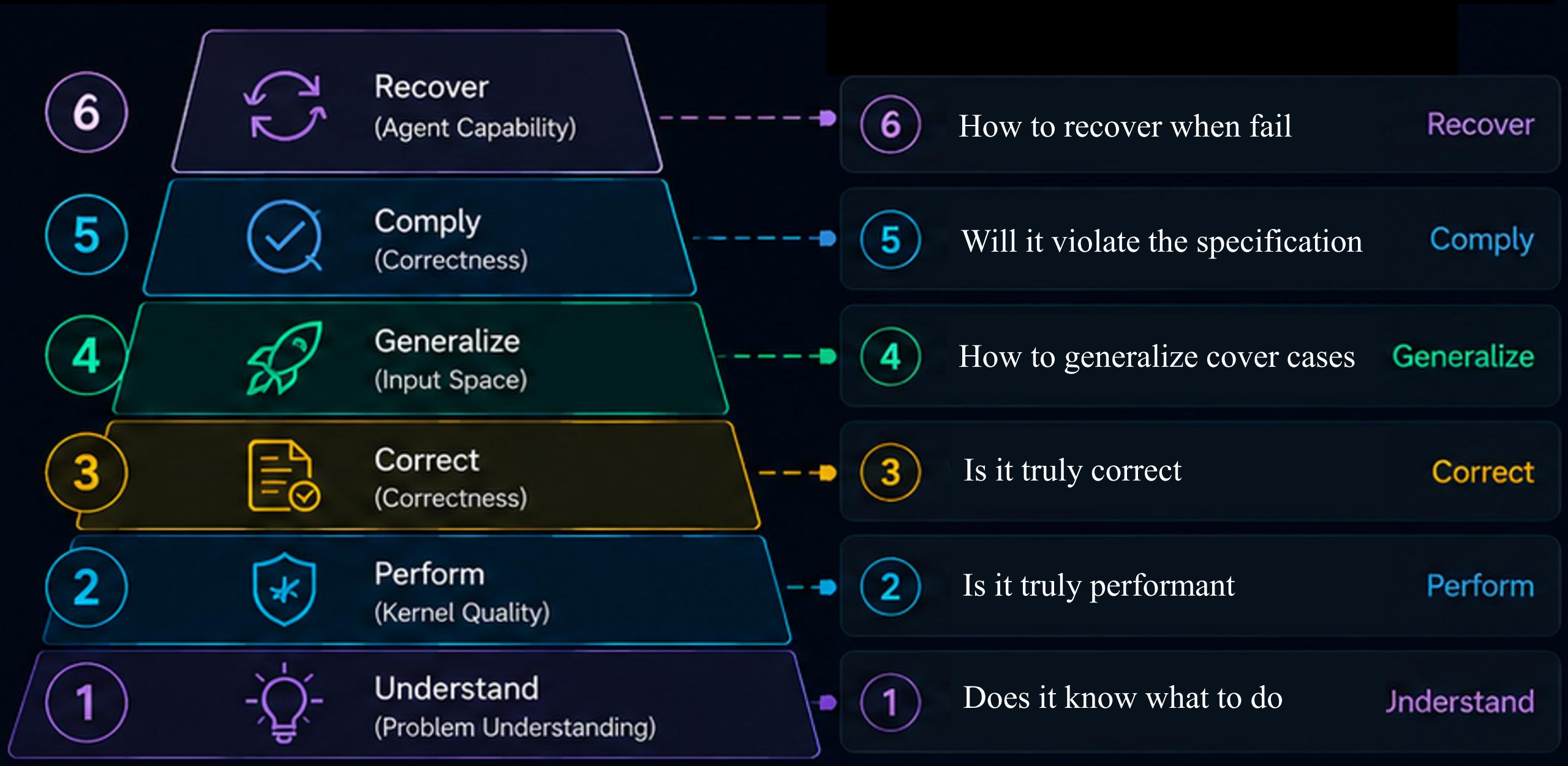
Kernel Generation

From generation to optimization

- Generate
- Compile
- Verify
- Benchmark
- Diagnose
- Optimize



Six key point of kernel generation



Capability One: You Talk, It Writes

北京智源人工智能研究院
BEIJING ACADEMY OF ARTIFICIAL INTELLIGENCE

Pure AI: Guessing from scratch

Error-prone, poor performance, requires repeated attempts

Hand-coded: Relying on experts

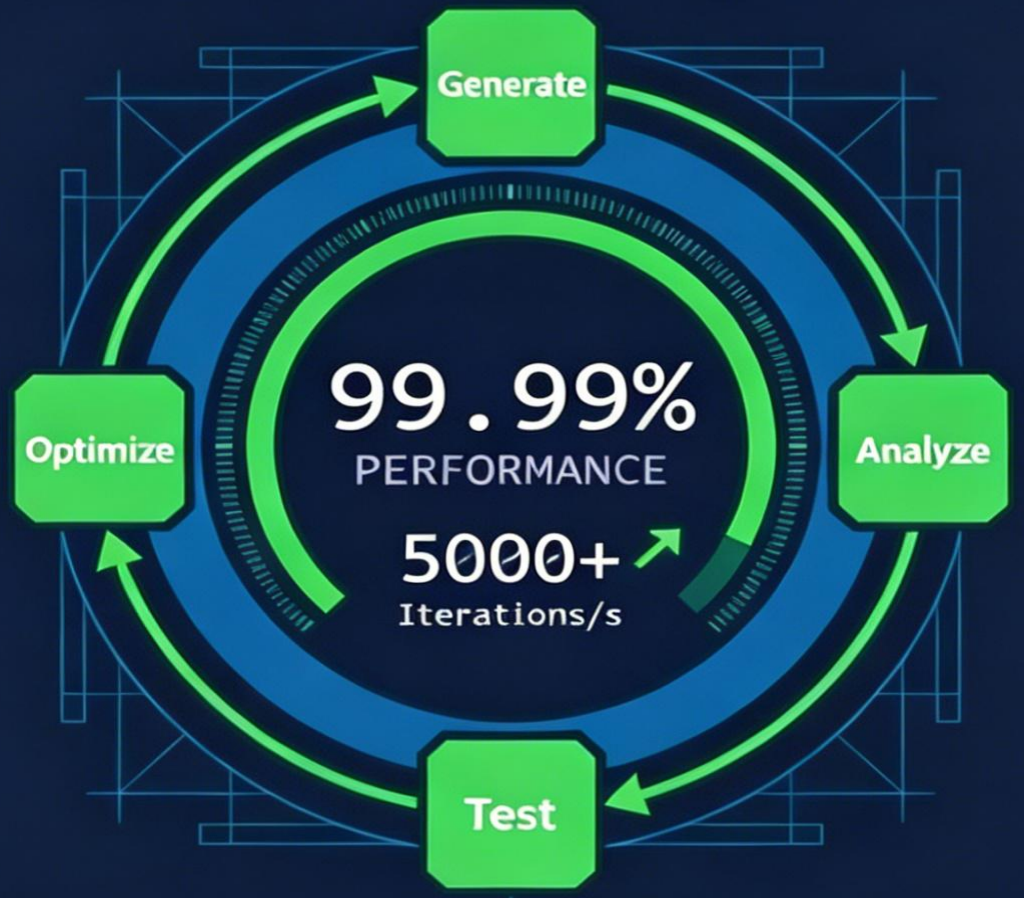
High cost, non-scalable poor performance, requires repeated attempts

KernelGen: Expert experience + AI automation

Successful on the first try with high performance

Capability Two: It Doesn't Just Write — It Pushes for Performance

Tireless Racing Tuner



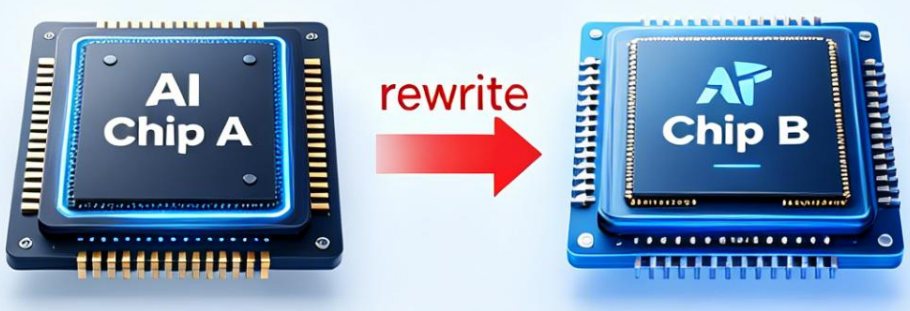
Automatically iterates multiple rounds without human participation.

Capability Three: Write Once, Run Anywhere

Strategically Significant Capability

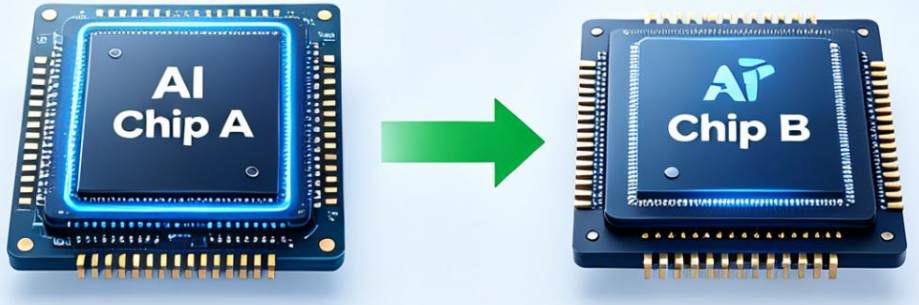
Traditional Approach

A kernel written for Chip A needs to be rewritten from scratch to run on Chip B



KernelGen 2.0

The system automatically migrates your kernel to a different chip



This is especially important for resource-constrained teams.

北京智源人工智能研究院
BEIJING ACADEMY OF ARTIFICIAL INTELLIGENCE



A top-tier AI chip like the NVIDIA H100 costs **over ten thousand dollars**.



Some teams simply **can't afford it**. In some regions, it's **hard to even reliably get access** to such hardware.

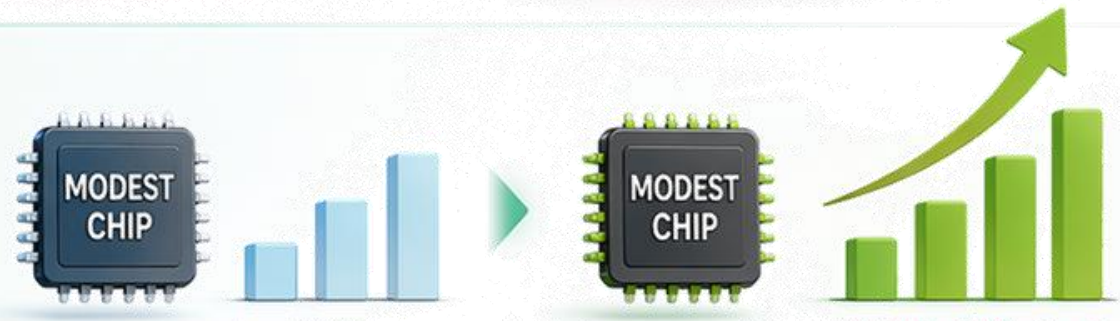


Cloud resources aren't cheap either — billed by the hour, **the costs add up fast**.



So, software-level efficiency becomes more important than the hardware itself.

If your kernels are efficient enough, even a modest chip can deliver solid results.



Poorly optimized kernel

Highly optimized kernel

That's exactly what KernelGen does —

it helps you get **the best possible performance** out of whatever chip you have.



Maximize performance on any hardware



Reduce costs, save resources



Work anywhere, on any infrastructure



More results with fewer limits

CAPABILITY FOUR

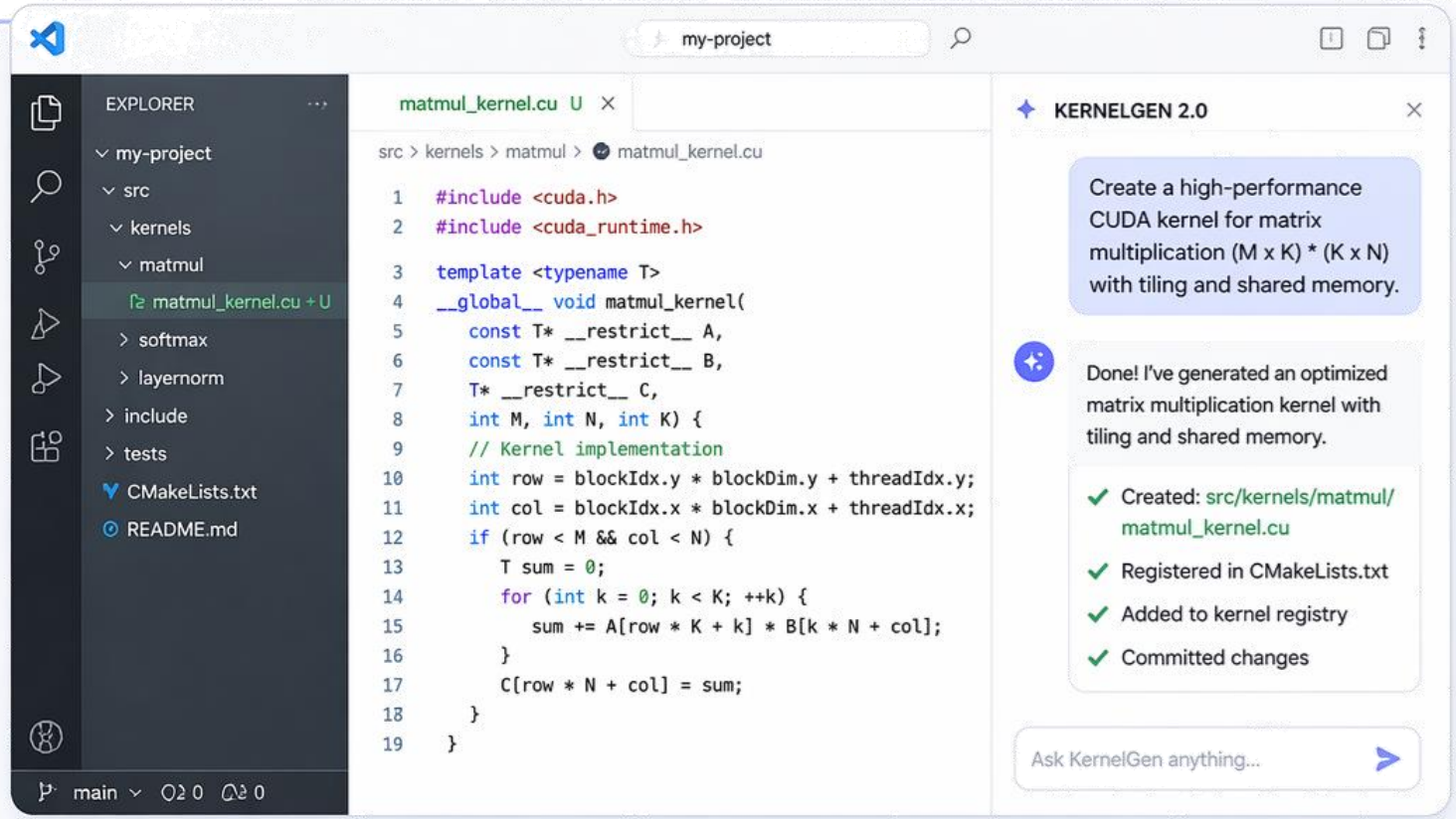
It Lives Inside Your Development Environment

北京智源人工智能研究院
BEIJING ACADEMY OF ARTIFICIAL INTELLIGENCE

KernelGen 2.0 isn't a standalone tool — it embeds directly into the coding environments developers use every day.

No window-switching, no copy-pasting.
Just say something in your coding environment, and it starts working.

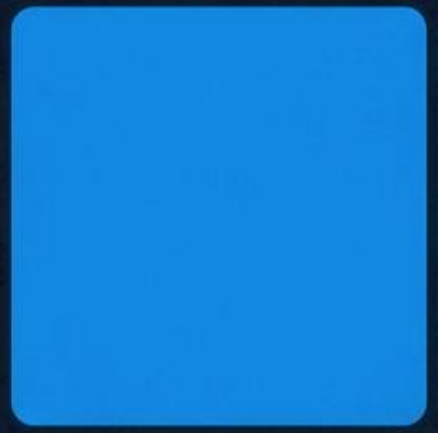
Generated code is automatically placed in the right directory, registered, and committed.



You describe what you need → KernelGen generates the code → Automatically placed in the right directory → Registered in the project → Committed to version control → Ready to build and run

Let the Data Speak — A Real Head-to-Head

Contestant #1



KernelGen 2.0

Contestant #2



A well-known AI coding assistant

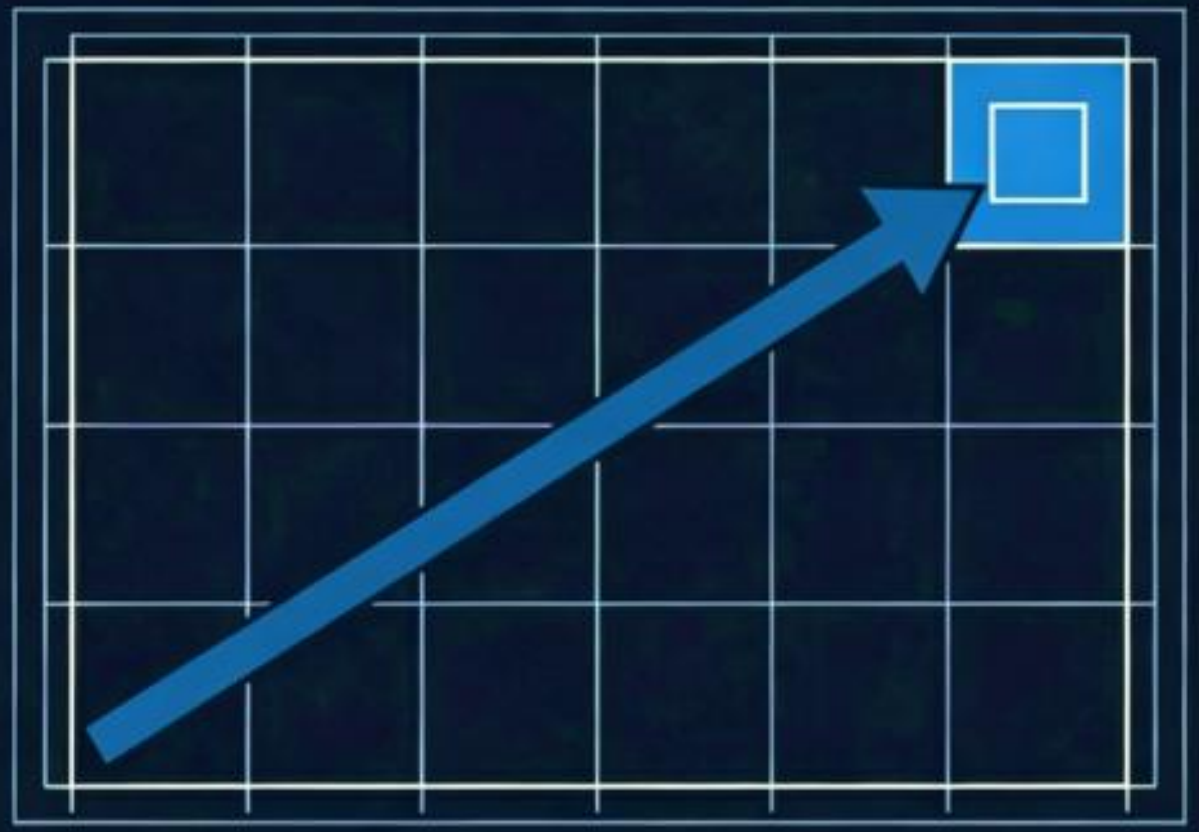
LayerNorm

It's a critical step in every layer of an AI model that keeps computations stable.
It needs to be both fast and accurate — a key path for performance optimization.

Metrics	KernelGen 2.0	AI coding assistant
Time to complete	14 Minutes	35 Minutes
Resources consumed	10K tokens	22K tokens
Accuracy	100% pass	100% pass
Runtime speed **1.14X** Faster than baseline		Runtime speed **0.64X** Slower than baseline

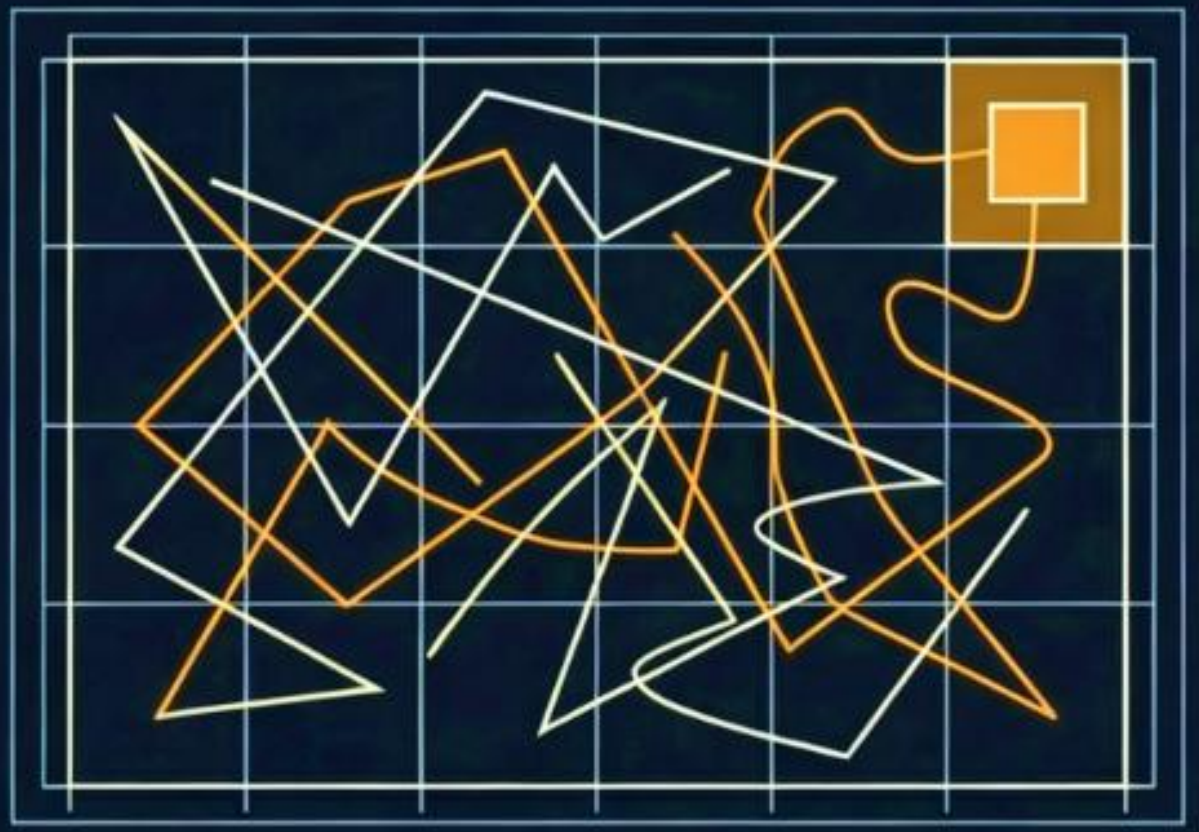
Half the time, half the resources, nearly twice the performance.

Why is there such a big gap?



KernelGen = Expert Experience × AI

Standing on the shoulders of experts,
experts, the direction is clear.



Pure AI = AI + Trial and Error

Starting from scratch to guess,
repeatedly consuming resources.

Why Does This Matter to You

Today we talked about KernelGen, but behind it is something much bigger

- AI is beginning to participate in the deepest layers of software engineering, becoming a powerful partner for engineers, from writing application code, to system code, to kernels.
- KernelGen is just the first step on this path.

So please remember three things

- Kernels are the invisible engine of the AI world.
- Engine is now being rewritten by AI itself.
- you, no matter where you are, can be part of it.


Trillions upon trillions of calculations, completed in a fraction of a second.
AI
Using Kernel written by AI itself



Thanks !

Website：https://kernelgen.flagos.io/
User Manual： https://docs.flagos.io/projects/kernelgen/en/latest/